Grafting, Not Rewiring: A Function-Preserving Stride-4 Level for a Pretrained YOLOv10n

and Its Interaction with Small-Target Label Assignment on VisDrone

[Author names, affiliations and e-mail addresses to be completed by the authors]

SYNTHETIC NUMBERS - GENERATOR TEST ONLY - NOT RESULTS

Abstract. A stride-4 (P2) prediction level is the standard remedy for small objects in UAV imagery, but it is usually added by re-wiring the neck of a COCO-pretrained detector, which discards much of the pretrained weights, and its benefit is rarely separated from label assignment, input resolution and evaluation protocol. We study these factors on VisDrone-DET2019 with YOLOv10n under one protocol (saved predictions, COCO AP, TIDE, paired bootstrap, 3 seeds, 40 epochs). P2-Graft appends a stride-4 branch that only *reads* the pretrained neck: the original levels are reproduced exactly at initialisation (max. output difference exactly 0 (bit-identical)) and all inheritable pretrained parameters are used, versus 53% for textbook re-wiring. An exact anchor analysis shows that 16.6% of the training boxes own no anchor centre on P3–P5 under classic task-aligned assignment (3.4% with P2), which the small-target window floor of the Ultralytics release removes by itself. Results: P2-Graft improves AP by 1.1 points (17.0→18.1); the gain of the stride-4 level is +1.4 AP under classic assignment and +1.1 AP with the window floor; naive re-wiring is 2.9 points below P2-Graft (1 seed); a baseline at higher resolution and matched GFLOPs is 0.5 points above P2-Graft (1 seed).

Keywords. small object detection, UAV imagery, YOLOv10, pretrained initialisation, label assignment, reproducible evaluation.

1. INTRODUCTION

Unmanned aerial vehicles are now routine sensing platforms for traffic monitoring, search and rescue and infrastructure inspection, and the detectors that run on them must be small and fast [21, 29, 7]. Aerial detection is hard mainly because objects are tiny: in the VisDrone-DET2019 training split (6471 images, 343,204 annotated objects) 60.5% of the boxes have an area below 32^2 px and 26.0% below 16^2 px at the original resolution; after the usual resize to a 640 px network input the median box is only 10.7×11.4 px [29].

The most common remedy is to add a stride-4 (P2) prediction level to a COCO-pretrained detector [30, 26, 16, 20]. In practice this is done by editing the neck, so that the bottom-up path and the detection heads receive different inputs than they were pretrained with. Frameworks then initialise the new model by copying every tensor whose name and shape happen to match, which keeps part of the weights, silently drops the rest, and can place a tensor in a layer whose role has changed. At the same time, the benefit of the extra level is entangled with three other factors that papers rarely control: (i) how small ground-truth boxes are assigned to anchors, (ii) the input resolution, and (iii) the evaluation protocol, including whether baselines are re-trained or quoted from other publications.

We therefore ask four pre-specified questions, answered under one training and evaluation pipeline:

- **H1** Does a stride-4 level improve a pretrained YOLOv10n on VisDrone when the library's default assignment is used?
- **H2** Does it matter *how* the pretrained weights are carried over (grafting vs. re-wiring)?
- **H3** Is the benefit of the stride-4 level smaller when the assigner already enlarges the candidate window of tiny boxes?
- **H4** At matched compute, is a stride-4 level a better use of FLOPs than a higher input resolution?

Contributions. (1) *P2-Graft*, a function-preserving way to extend a pretrained detector with a stride-4 level, together with an explicit role-based weight-transfer rule and transfer accounting (Section 3). (2) An exact anchor-candidate analysis of VisDrone that quantifies how many boxes cannot be assigned at all under classic task-aligned assignment and how a P2 level or a window floor changes this. (3) A controlled study with a matched baseline trained by us, a 2×2 factorial over level and assignment, negative controls (naive re-wiring, removing P5, random initialisation) and a compute-matched resolution baseline, evaluated from saved predictions with COCO AP, TIDE and an image-level paired bootstrap. All numbers are produced by one open notebook, and the text of Section 5 is generated from its outputs.

Outcome in brief. H1 supported. H2 inconclusive. H3 supported. H4 inconclusive. Details and caveats are in Sections 5 and 6.

2. RELATED WORK

Small objects and high-resolution pyramid levels. Feature pyramids [13, 15] are the standard answer to scale variation, and a fourth prediction head fed by high-resolution backbone features is a frequent remedy for tiny objects in drone imagery [30, 16, 20]; QueryDet [26] reduces the cost of high-

resolution features with sparse queries, and slicing at inference [1] trades latency for recall. Surveys and benchmarks document that objects below 32×32 px remain the dominant failure case [18, 19, 29, 7, 23, 5].

YOLO variants for aerial images. Many papers modify YOLOv8/v10/v11 with attention, lightweight convolutions or extra heads and compare against numbers taken from other publications [17, 28, 10, 25, 16, 20]. Such tables combine different training lengths, augmentations, input sizes and evaluation code, so a gain over a quoted baseline cannot be attributed to the proposed module; we avoid this by training every compared model ourselves, with identical code, data order seeds and evaluation.

Label assignment for tiny objects. Task-aligned assignment [9], in the form used by YOLOv8-style detectors, chooses for every ground-truth box the top-k anchors among those whose centres lie inside the box. For boxes smaller than the grid stride this set can be empty, which motivated Gaussian receptive-field assignment [24] and Wasserstein-based similarity [22]. The Ultralytics release used in this study [11] enlarges the candidate window of very small boxes to a minimum side length (Section 3.4); we treat this *window floor* as an experimental factor.

Growing a pretrained network. Net2Net [4] introduced function-preserving transformations that widen or deepen a trained network without changing its output, and ControlNet [27] attaches new trainable branches to a frozen network so that the augmented model starts exactly at the original. P2-Graft applies the same principle to the *detection pyramid*: the new level only consumes existing activations.

Evaluation practice. We use COCO AP [14] with the detection cap raised to the VisDrone density, TIDE error analysis [2] on saved predictions, multiple seeds [3] and an image-level bootstrap [8].

3. METHOD

3.1. Setting

We use the nano YOLOv10 detector [21] as implemented in Ultralytics 8.4.150 [11]: a CSP-style backbone B (stem, C2f stages, SCDown, SPPF, PSA), a PAN-style neck N producing features p_3, p_4, p_5 (strides 8/16/32) and one detection head H_ℓ per level with a one-to-many and a one-to-one branch. Training uses the end-to-end loss of this release, which mixes a one-to-many loss (top-k task-aligned assignment) and a one-to-one loss, with weights that move from 0.8/0.2 to 0.1/0.9 over the schedule; inference uses the NMS-free one-to-one branch unless stated otherwise. Box regression uses distribution focal loss [12]. The COCO-pretrained checkpoint has 80 classes; the final class-logit convolutions of a 10-class model are reinitialised, and rows whose class name matches a COCO class (bicycle, car, truck, bus) are copied by name.

3.2. P2-Graft

Let b_2 be the stride-4 backbone feature (output of the first C2f stage) and p_3 the pretrained P3 neck feature. P2-Graft adds

$$p_2 = G_\phi\big([\mathrm{Up}_{2\times}(p_3), b_2]\big), \qquad y_2 = H^{(2)}_\psi(p_2), \tag{1}$$

where G_ϕ is a neck block of the same type and width as the P3 block and $H^{(2)}_\psi$ is a detection head built like the others (Figure 1). The extended model outputs $\{y_3, y_4, y_5, y_2\}$; the new level is appended *last*, so the parameter names of the existing levels do not change.

Proposition 1 (exact preservation) *If the parameters of B, N and H_3, H_4, H_5 equal those of the pretrained detector, the outputs y_3, y_4, y_5 of the grafted model equal those of the pretrained detector for every input, whatever ϕ, ψ are.*

Proof. G_ϕ and $H^{(2)}_\psi$ consume p_3 and b_2 and produce only new tensors; no existing layer reads their outputs. □ We also verified this numerically on random inputs with the real checkpoint: the maximum absolute difference over the neck features and over the raw outputs of all four prediction branches (one-to-many/one-to-one × box/class) of the three original levels is exactly 0 (bit-identical). The statement concerns the *starting point*: all parameters, pretrained ones included, are fine-tuned jointly, so gradients of the new level reach the shared layers.

3.3. Transfer accounting

Let Θ_{inh} be the parameters of the 80-class checkpoint that a 10-class model of the *same* architecture can inherit (everything except the shape-dependent class-logit convolutions). For a variant and a transfer rule we report the fraction of Θ_{inh} that is copied, and the fraction that is copied into a layer whose entire upstream sub-graph (types, arguments, wiring) is identical to the one in the pretrained model (*role-aligned*). Two rules are compared: *key matching*, which is what Ultralytics does by default (same tensor name and shape), and a *role-based map* written per variant (for P2-Graft it only renames the detection module's index, $23 \to 26$; for the P5-removal variant it additionally drops the P5 neck and the P5 branch of the head).

3.4. Anchor assignability and the window floor

In Ultralytics' task-aligned assigner a ground-truth box $b = (x_1, y_1, x_2, y_2)$ can only receive positives from anchors whose centres $((i+\frac{1}{2})s, (j+\frac{1}{2})s)$ lie strictly inside the box. The release used here first raises every side shorter than $w_{\min}$ to $w_{\min}$ (around the box centre) for this test, where $w_{\min}$ defaults to the second entry of the model's stride list (16 px for P3–P5); we call $w_{\min}$ the *window floor* and set it explicitly in every run. With $\tilde{b}$ the floored box and $\mathscr{S}$ the set of strides, the number of candidates is

$$N(b) = \sum_{s \in \mathscr{S}} \#\{(i,j) : \tilde{x}_1 < (i+\tfrac{1}{2})s < \tilde{x}_2,\ \tilde{y}_1 < (j+\tfrac{1}{2})s < \tilde{y}_2\}, \tag{2}$$

and $N(b) = 0$ means the box can never become a positive. We evaluate $N(b)$ exactly on all training boxes (Section 5.1).

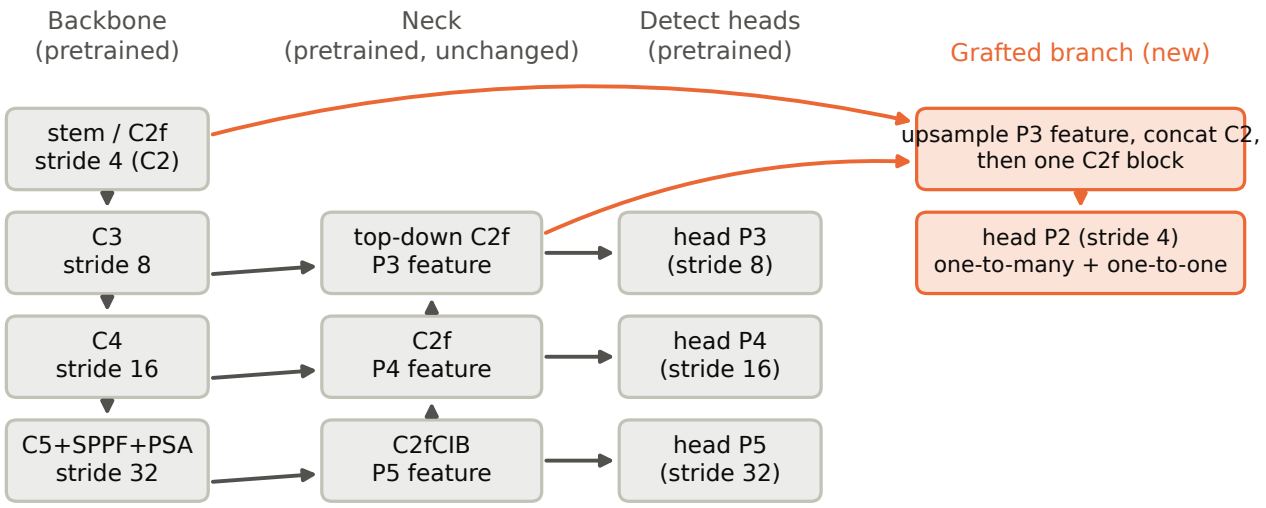


Figure 1: P2-Graft. Grey modules are the pretrained detector and are left untouched; orange modules are new. The new branch reads the P3 neck feature and the stride-4 backbone feature and adds a fourth prediction level.

3.5. Systems compared

Baseline: YOLOv10n with COCO initialisation (role map = identity). **P2-Graft** (ours). **Naive P2**: the textbook YOLOv8-P2-style re-wiring (P2 output feeds a new bottom-up path), initialised by key matching. **P2→P5**: a P2-Graft whose P5 level and P5 neck are removed. **Random init**: baseline and P2-Graft without pretrained weights. **Resolution baseline**: the baseline trained and tested at a higher input size chosen to match the GFLOPs of P2-Graft. Each can be trained with the library-default window floor (16 px) or with classic assignment (floor 0).

4. EXPERIMENTAL PROTOCOL

Data. VisDrone-DET2019 [29, 6]: 6,471 training, 548 validation and 1,610 test-dev images, ten classes. Ignored regions and "others" are not used as objects. All models are trained on the full training split with one fixed recipe, *without* any hyperparameter search and *without* checkpoint selection (the last-epoch EMA weights are evaluated), so validation and test-dev are both untouched held-out sets. Images are resized once (INTER_AREA) to the training long side and boxes are mapped back to the original resolution for evaluation.

Training. 40 epochs from the COCO checkpoint, SGD (lr 0.01, momentum 0.937, weight decay 5×10^{-4}, linear decay to $0.01\times$), 3 warm-up epochs, batch 32 with gradient accumulation to a nominal batch of 64, mosaic augmentation switched off for the last 10 epochs, mixed precision, EMA, input 640 px (unless stated), the Ultralytics default augmentation otherwise. The schedule is deliberately short (the budget of one GPU session); absolute numbers are therefore lower than 300-epoch results reported in the literature and must not be compared with them.

Hardware and software. SYNTHETIC GPU, PyTorch x, Ultralytics 8.4.150, 4 CPU threads. Latency is the median of 200 batch-1 forward passes of the fused deploy graph in FP16.

Evaluation. Predictions are produced with confidence threshold 0.001 and at most 500 detections per image (VisDrone images contain up to 902 objects), saved, and scored against the *original-resolution* boxes with COCO AP (IoU 0.50:0.95), AP_{50}, AP in the COCO area bins and two finer bins ($< 16^2$ and 16^2–32^2 px). The primary head is the NMS-free one-to-one branch. Secondary results use (i) the one-to-many branch with NMS, (ii) VisDrone's ignore regions turned into per-class crowd regions, and (iii) larger test sizes. TIDE [2] is computed from the same saved predictions.

Statistics. Main conditions are repeated with 3 seeds (initialisation of new layers, augmentation and data order). We report mean ± sample standard deviation, paired per-seed differences, and a paired image-level bootstrap (– resamples of the validation images, seed-averaged AP) for differences between conditions. The primary metric of H1–H4 is AP (IoU 0.50:0.95) on VisDrone-val. With so few seeds, p-values are not reported as evidence; single-seed controls are labelled as such.

5. RESULTS

5.1. How many boxes can be assigned at all?

At 640 px the median training box is 10.7×11.4 px, 45.7% of the boxes have a side below 8 px and 15.5% below 4 px. Under classic task-aligned assignment 16.6% of the boxes own no anchor centre on P3–P5 (47.8% of those with $\sqrt{wh} < 8$ px); a P2 level lowers the overall share to 3.4%, and a window floor of 8 or 16 px reduces it to 0.18% / 0.00% even without P2 (Table 1, Figure 2). The floor achieves this by admitting anchors that lie *outside* the true box (candidates per box: 8.6 → 11.3). The two mechanisms therefore address the same

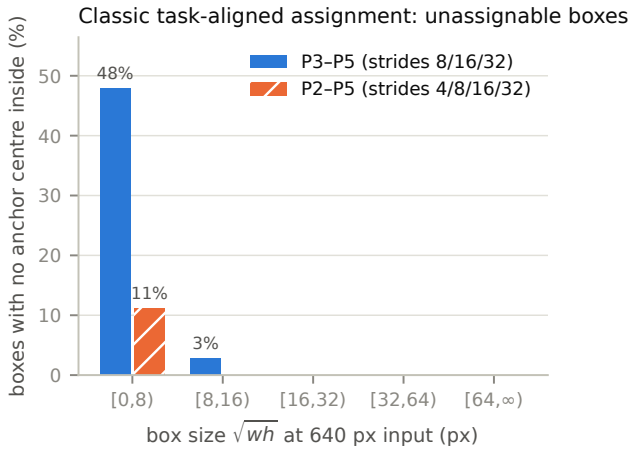


Figure 2: Boxes without any candidate anchor under classic task-aligned assignment, by box size (VisDrone-train, 640 px input).

problem, which motivates H3.

5.2. Exactness and transfer coverage

The grafted model reproduces the three original levels exactly (max. difference exactly 0 (bit-identical)) and inherits 100% of Θ_{inh} under the role-based map, whereas key matching copies only 70.5% for the same architecture (the detection module's index shifts) and 52.9% for the naive P2 re-wiring (Table 2); in the P2→P5 variant key matching additionally copies 18.0% of Θ_{inh} into layers whose role has changed.

5.3. Main comparison

P2-Graft improves AP by 1.1 points (17.0 → 18.1); AP_{50} changes by +1.4 points and AP on objects below 16^2 px by +0.6. The per-seed differences are +1.1, +1.2, +0.9 (the sign is identical across the 3 paired seeds). The extra level costs 12.7 vs 6.6 deploy GFLOPs (×1.92) and 6.20 vs 4.10 ms median latency, with 2.39 vs 2.27 M deploy parameters. See Table 3 and Figure 3.

5.4. Stride-4 level × assignment

Under classic task-aligned assignment the stride-4 level adds +1.4 AP; with the library's small-target window floor it adds +1.1 AP, i.e. an interaction contrast of −0.36. Hypothesis H3 is supported (basis: same sign in all 3 paired seeds (no bootstrap interval available)); see Table 4 and Figure 4.

5.5. Controls: re-wiring, removing P5, random initialisation

Re-wiring the neck instead of grafting (key-matched transfer; key matching inherits 53% of the inheritable pretrained parameters) gives 15.2 AP, 2.9 points below P2-Graft (1 seed). Removing P5 while keeping the grafted P2 level gives 17.3 AP (−0.8 vs. P2-Graft) and 32.9 AP_L (−1.4), which tests the earlier suggestion that dropping the stride-32 head hurts large vehicles. Without pretrained weights P2-Graft reaches 6.0 AP after 40 epochs (−12.1 vs. the pretrained model). Without pretrained weights the baseline reaches 6.6 AP after 40 epochs (−10.4 vs. the pretrained model). Controls other than the main pair use a single seed unless Table 3 lists more; the seed-to-seed standard deviation of the main conditions gives the scale of noise.

5.6. Compute-matched comparison with input resolution

A baseline trained and tested at higher resolution (896 px, 12.9 GFLOPs) reaches 18.6 AP, 0.5 points above P2-Graft at 12.7 GFLOPs. Hypothesis H4 is inconclusive (basis: insufficient evidence (1 paired seed, no interval)); see Figure 6.

Table 5 evaluates the models trained at 640 px at larger test sizes (800 and 960 px) without retraining, which spends the same extra compute at inference time only.

5.7. Error analysis

TIDE attributes the AP_{50} lost relative to an oracle to error types (Table 6). Change from the baseline to P2-Graft in points of AP_{50} (negative = fewer errors): Cls +0.04, Loc 0.00, Both 0.00, Dupe 0.00, Bkg 0.00, Miss −0.80.

Figure 8 shows the per-class change in AP_{50}.

5.8. Test-dev and protocol sensitivity

On the 1,610 test-dev images the AP change is +1.0 (15.8 → 16.8), and the AP_{50} change is +1.3. Turning VisDrone's ignore regions into crowd regions raises AP_{50} from 30.1 to 31.0 (baseline) and from 31.5 to 32.4 (P2-Graft); the difference between the two models changes from +1.4 to +1.4. Using the one-to-many branch with NMS instead of the NMS-free branch gives 17.3 (baseline) and 18.4 AP (P2-Graft). Shortcuts in the evaluation pipeline matter as much as architecture changes: scoring only detections with confidence ≥ 0.25 (the operating point of precision/recall columns) lowers AP_{50} from 30.1 to 24.1 for the baseline and from 31.5 to 25.2 for P2-Graft, and the difference between the models becomes +1.1 instead of +1.4; capping the output at 100 detections per image (the COCO default) gives 30.0 and 31.3 although VisDrone images contain up to hundreds of objects. As an independent cross-check, Ultralytics' own validator (rectangular batches, its own matching rule) gives AP_{50} 30.4 / 31.8 for the baseline / P2-Graft, a difference of +1.4 against +1.4 with the COCO evaluator; absolute values of different evaluators can differ by about a point, so only differences measured with one evaluator are compared in this paper.

6. DISCUSSION

Reading the evidence. Verdicts are computed mechanically: "supported" or "contradicted" requires that the 95% image-bootstrap interval of the seed-averaged difference excludes zero *and*, with several paired seeds, the same sign in every pair; everything else is "inconclusive". Single-seed verdicts say so, because image-level intervals capture test-set sampling, not training variance. All verdicts describe this model scale, dataset and schedule.

Limitations. (i) Short fine-tuning budget (40 epochs); the new level starts from random weights, so a longer schedule may favour it more than reported here. (ii) One training dataset

Table 1: Share of VisDrone-train boxes (343k, long side resized to 640 px) with no anchor centre inside the box, i.e. unable to become a positive of a task-aligned assigner, for different pyramids and window floors. Exact counts at the true positions; average of the true placement and four random sub-grid shifts.

Levels	floor 0 (%)	floor 8 (%)	floor 16 (%)	cand./box (floor 0)	cand./box (floor 16)
P3–P5 (8/16/32)	16.6	0.2	0.0	8.6	11.3
P2–P5 (4/8/16/32)	3.4	0.0	0.0	34.8	45.6

Table 2: Pretrained parameters inherited by each variant (100% = everything a 10-class baseline can inherit from the 80-class COCO checkpoint). 'Key matching' is Ultralytics' default (same tensor name and shape); a copied tensor is role-aligned only if the whole sub-graph feeding that layer is unchanged.

Variant / transfer rule	copied (%)	role-aligned (%)	params, train (M)	params, deploy (M)	GFLOPs
baseline (reference)	100.0	100.0	2.71	2.27	6.6
P2-Graft, role-based map	100.0	100.0	2.92	2.39	12.7
P2-Graft, key matching	70.5	70.5	2.92	2.39	12.7
naive P2, key matching	52.9	52.9	2.83	2.35	10.7
P2 replaces P5, role-based map	73.9	73.9	2.19	1.88	12.2
P2 replaces P5, key matching	77.0	59.0	2.19	1.88	12.2

(VisDrone) and one detector scale (nano); a second detector family (YOLO26n), if it was run, appears only in the run table of Appendix A, and no cross-dataset evaluation is performed. (iii) Controls other than the main pair are single-seed unless Table 3 lists more. (iv) The window floor is a property of the Ultralytics release used; other frameworks differ. (v) Latency is measured on one GPU, not on embedded hardware. (vi) The main protocol follows the YOLO literature (no ignore-region rule); the ignore-region variant is reported. (vii) No state-of-the-art claim is made.

Recommendations (independent of the accuracy ranking). When extending a pretrained detector with a pyramid level, report (a) how many pretrained parameters were carried over and where, (b) the assigner's treatment of boxes smaller than the stride, and (c) a baseline trained under the same pipeline plus a resolution baseline of equal FLOPs.

Ethical considerations. Aerial detectors have dual uses (traffic monitoring, search and rescue, but also surveillance). This study uses a public research benchmark, evaluates coarse object categories only, and releases no model or tool aimed at identifying individuals.

Reproducibility. The study is one marimo notebook (`graft_p2_study.py`, version 1.0.0, SHA-256 0107fcdb735603c5…) that downloads VisDrone, trains all runs, evaluates from saved predictions and writes this paper (software and hardware: Section 4; Python 3). Code and saved predictions: *[URL to be added]*. Ultralytics is released under AGPL-3.0; VisDrone is distributed for research use.

7. CONCLUSION

P2-Graft improves AP by 1.1 points (17.0 → 18.1); AP_{50} changes by +1.4 points and AP on objects below 16^2 px by +0.6. The per-seed differences are +1.1, +1.2, +0.9 (the sign is identical across the 3 paired seeds). Under classic task-aligned assignment the stride-4 level adds +1.4 AP; with the library's small-target window floor it adds +1.1 AP, i.e. an interaction contrast of −0.36. P2-Graft shows that an added pyramid level can start exactly at the pretrained detector; whether it should be preferred over re-wiring, over a window floor, or over a higher input size is answered by Tables 3–7 for this setting, and the released notebook makes the same questions cheap to ask for other detectors and datasets.

A. ALL RUNS

REFERENCES

[1] Fatih Cagatay Akyon, Sinan Onur Altinuc, and Alptekin Temizel. Slicing aided hyper inference and fine-tuning for small object detection. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pages 966–970, 2022.

[2] Daniel Bolya, Sean Foley, James Hays, and Judy Hoffman. TIDE: A general toolbox for identifying object detection errors. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 558–573, 2020.

[3] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[4] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] Gong Cheng, Xiang Yuan, Xiwen Yao, Kebing Yan, Qinghua Zeng, Xingxing Xie, and Junwei Han. Towards large-scale small object detection: Survey and benchmarks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):13364–13387, 2023.

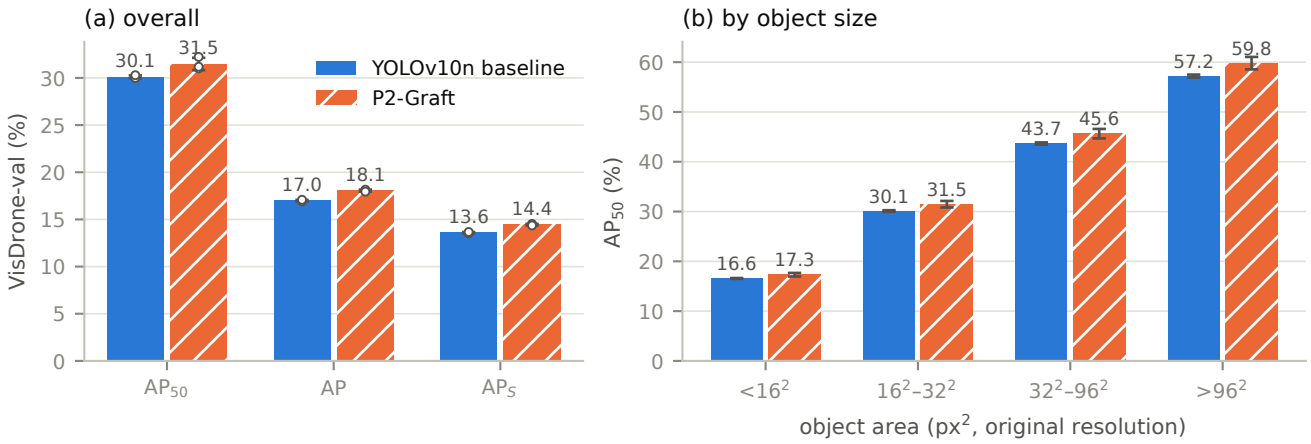


Figure 3: Baseline vs. P2-Graft on VisDrone-val: (a) overall metrics and (b) AP_{50} by object area. Bars: mean over seeds; error bars: sample std; dots: individual seeds.

Table 3: Main results on VisDrone-DET2019 validation (and test-dev for the last column), mean ± std over n seeds, 40-epoch fine-tuning from the COCO checkpoint. Parameters and GFLOPs are for the deploy graph (fused, one-to-one head only) at the evaluated resolution; latency is the median batch-1 FP16 forward pass on the GPU of Section 4.

Model	n	Par. (M)	GFLOPs	ms	AP_{50}	AP	AP_S	$AP_{<16^2}$	AP^{test}
YOLOv10n baseline	3	2.27	6.6	4.10	30.1 ± 0.2	17.0 ± 0.1	13.6 ± 0.1	9.3 ± 0.0	15.8 ± 0.1
P2-Graft (ours)	3	2.39	12.7	6.20	31.5 ± 0.7	18.1 ± 0.1	14.4 ± 0.1	9.9 ± 0.1	16.8 ± 0.1
naive P2 re-wiring	1	2.35	10.7	5.50	27.0	15.2	12.2	8.4	14.1
P2 replaces P5	1	1.88	12.2	5.80	30.2	17.3	13.8	9.5	16.1
P2-Graft, random init	1	2.39	12.7	6.20	12.0	6.0	4.8	3.3	5.6
baseline, random init	1	2.27	6.6	4.10	13.0	6.6	5.3	3.6	6.1
baseline @ 896 px	1	2.27	12.9	6.60	32.0	18.6	14.9	10.2	17.3

[6] Dawei Du et al. VisDrone-DET2019: The vision meets drone object detection in image challenge results. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

[7] Dawei Du, Yuankai Qi, Hongyang Yu, Yifan Yang, Kaiwen Duan, Guorong Li, Weigang Zhang, Qingming Huang, and Qi Tian. The unmanned aerial vehicle benchmark: Object detection and tracking. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 370–386, 2018.

[8] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[9] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R. Scott, and Weilin Huang. TOOD: Task-aligned one-stage object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3490–3499, 2021.

[10] Zhuolun He, Rui She, Bo Tan, Jiajian Li, and Xiaolong Lei. SSCW-YOLO: A lightweight and high-precision model for small object detection in UAV scenarios. *Drones*, 10(1):41, 2026.

[11] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO. https://github.com/ultralytics/ultralytics, 2026. version 8.4.150, AGPL-3.0.

[12] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.

[14] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[15] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. Path aggregation network for instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8759–8768, 2018.

Table 4: Stride-4 level × small-target window floor of the task-aligned assigner (VisDrone-val, mean ± std over seeds).

Assignment	AP base	AP graft	Δ	AP_S base	AP_S graft	Δ
library default (floor 16 px)	17.0 ± 0.1	18.1 ± 0.1	+1.1	13.6 ± 0.1	14.4 ± 0.1	+0.8
classic TAL (floor 0)	16.0 ± 0.0	17.4 ± 0.1	+1.4	12.8 ± 0.0	14.0 ± 0.1	+1.1

Table 5: Test-time input size without retraining (extra sizes: first seed only; GFLOPs scale with the pixel count).

Model (trained at 640 px)	AP @ 640 / 800 / 960	AP_S @ 640 / 800 / 960	GFLOPs @ 640 / 800 / 960
YOLOv10n baseline	17.0 / 18.2 / 18.7	13.6 / 14.5 / 14.9	6.6 / 10.3 / 14.8
P2-Graft	18.1 / 19.3 / 19.9	14.4 / 15.5 / 15.9	12.7 / 19.8 / 28.6

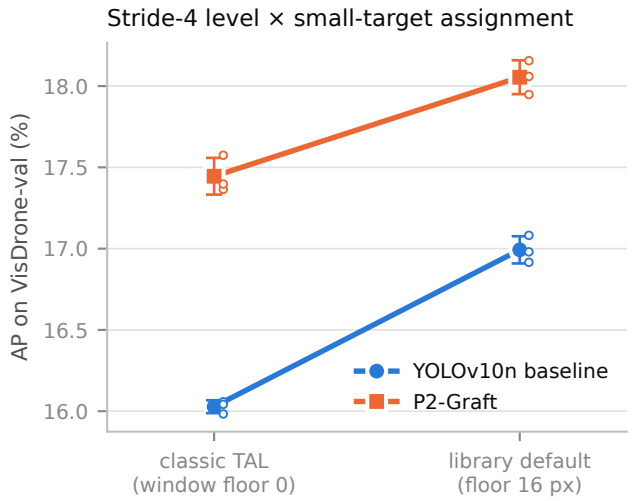


Figure 4: Interaction between the stride-4 level and the window floor of the assigner.

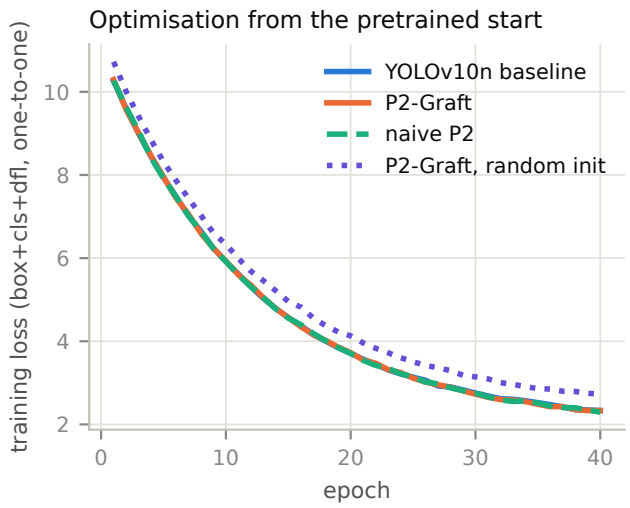


Figure 5: Training loss per epoch (box + class + DFL, one-to-one branch) of seed-0 runs; the pretrained start is compared with random initialisation.

[16] Yingqi Lyu, Ting Zhang, Xin Li, An Liu, and Gang Shi. LightUAV-YOLO: A lightweight object detection model for unmanned aerial vehicle image. *Journal of Supercomputing*, 81:172, 2025.

[17] Jianqiang Mei and Wei Zhu. BGF-YOLOv10: Small object detection algorithm from unmanned aerial vehicle perspective based on improved YOLOv10. *Sensors*, 24(21):6911, 2024.

[18] Mahya Nikouei, Bita Baroutian, Shahriar Nabavi, Fatemeh Taraghi, Amirreza Aghaei, Aydin Sajedi, and Mohsen Ebrahimi Moghaddam. Small object detection: A comprehensive survey on challenges, techniques and real-world applications. *Intelligent Systems with Applications*, 27, 2025.

[19] Aref Miri Rekavandi, Shima Rashidi, Farid Boussaid, Stephen Hoefs, Emre Akbas, and Mohammed Bennamoun. Transformers in small object detection: A benchmark and survey of state-of-the-art. *arXiv preprint arXiv:2309.04902*, 2023.

[20] Chen Song and Wei Ge. RDD-YOLO: An improved small object detection model for UAV images based on YOLO11. In *Proceedings of the International Conference on Aerospace System Science and Engineering*. IEEE, 2025.

[21] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. YOLOv10: Real-time end-to-end object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] Jinwang Wang, Chang Xu, Wen Yang, and Lei Yu. A normalized Gaussian Wasserstein distance for tiny object detection. *arXiv preprint arXiv:2110.13389*, 2021.

[23] Jinwang Wang, Wen Yang, Haowen Guo, Ruixiang Zhang, and Gui-Song Xia. Tiny object detection in aerial images. In *Proceedings of the International Conference on Pattern Recognition (ICPR)*, pages 3791–3798, 2021.

[24] Chang Xu, Jinwang Wang, Wen Yang, Huai Yu, Lei Yu, and Gui-Song Xia. RFLA: Gaussian receptive field based label assignment for tiny object detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 526–543, 2022.

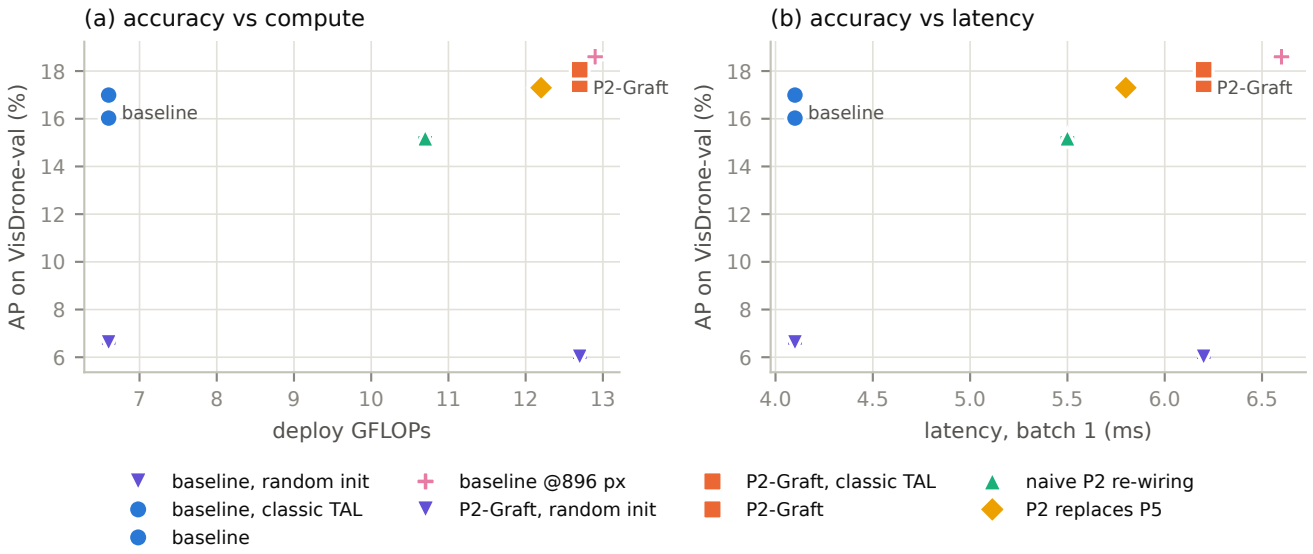


Figure 6: Accuracy versus deploy GFLOPs (a) and versus measured latency (b); one marker per condition (mean over seeds).

Table 6: TIDE ΔAP_{50} (points) computed from the saved VisDrone-val predictions (500 detections per image, boxes as $[x, y, w, h]$). Lower is better.

Error type	baseline	P2-Graft	Δ
Cls	3.00 ± 0.12	3.04 ± 0.09	$+0.04$
Loc	4.00 ± 0.00	4.00 ± 0.00	$+0.00$
Both	0.80 ± 0.00	0.80 ± 0.00	$+0.00$
Dupe	0.30 ± 0.00	0.30 ± 0.00	$+0.00$
Bkg	2.00 ± 0.00	2.00 ± 0.00	$+0.00$
Miss	6.30 ± 0.00	5.50 ± 0.00	-0.80
FalsePos	12.00 ± 0.00	12.00 ± 0.00	$+0.00$
FalseNeg	20.00 ± 0.00	20.00 ± 0.00	$+0.00$

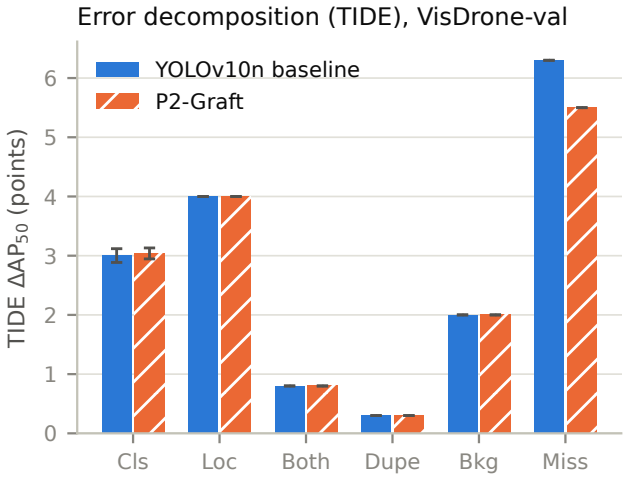


Figure 7: TIDE error decomposition on VisDrone-val.

[25] Hui Yan, Xiangzhi Kong, Tetsuya Shimada, and Hiroyuki Tomiyama. TOE-YOLO: Accurate and efficient detection of tiny objects in UAV imagery. *Journal of Real-Time Image Processing*, 22:194, 2025.

[26] Chenhongyi Yang, Zehao Huang, and Naiyan Wang. QueryDet: Cascaded sparse query for accelerating high-resolution small object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13668–13677, 2022.

[27] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3836–3847, 2023.

[28] Yang Zhang, Xin Chen, Shuo Sun, Hui You, Yue Wang, Jian Lin, and Jing Wang. Vehicle detection in drone aerial views based on lightweight OSD-YOLOv10. *Scientific Reports*, 15:25155, 2025.

[29] Pengfei Zhu, Longyin Wen, Dawei Du, Xiao Bian, Heng Fan, Qinghua Hu, and Haibin Ling. Detection and tracking meet drones challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7380–7399, 2022.

[30] Xingkui Zhu, Shuhao Lyu, Xiao Wang, and Qi Zhao. TPH-YOLOv5: Improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 2778–2788, 2021.

Table 7: Pre-specified hypotheses and what the collected runs support (primary metric: AP on VisDrone-val).

	Hypothesis	Verdict	Basis
H1	stride-4 level improves AP (library-default assignment)	supported	same sign in all 3 paired seeds (no bootstrap interval available)
H2	grafting transfers better than re-wiring	inconclusive	insufficient evidence (1 paired seed, no interval)
H3	benefit of the level is smaller when the window floor is active	supported	same sign in all 3 paired seeds (no bootstrap interval available)
H4	at matched GFLOPs the level beats higher resolution	inconclusive	insufficient evidence (1 paired seed, no interval)

Table 8: Every finished run (one row per seed).

Run	val AP_{50}	val AP	val AP_S	test AP_{50}	test AP	train (h)
v10-base-scratch-s0	13.0	6.6	5.3	12.0	6.1	0.67
v10-base-f0-s0	27.9	16.1	12.8	25.7	14.9	0.67
v10-base-f0-s1	28.9	16.0	12.8	26.6	14.9	0.67
v10-base-f0-s2	28.5	16.0	12.8	26.2	14.9	0.67
v10-base-f16-s0	30.0	17.0	13.6	27.6	15.8	0.65
v10-base-f16-s1	30.0	16.9	13.5	27.6	15.7	0.66
v10-base-f16-s2	30.3	17.1	13.7	27.9	15.9	0.68
v10-base-res896-s0	32.0	18.6	14.9	29.4	17.3	0.68
v10-graft-scratch-s0	12.0	6.0	4.8	11.0	5.6	0.67
v10-graft-f0-s0	30.7	17.4	13.9	28.3	16.1	0.67
v10-graft-f0-s1	31.0	17.6	14.1	28.5	16.3	0.67
v10-graft-f0-s2	31.1	17.4	13.9	28.6	16.2	0.67
v10-graft-f16-s0	31.0	18.1	14.4	28.5	16.8	0.68
v10-graft-f16-s1	32.2	18.2	14.5	29.6	16.9	0.66
v10-graft-f16-s2	31.2	17.9	14.4	28.7	16.7	0.66
v10-naive-key-s0	27.0	15.2	12.2	24.8	14.1	0.66
v10-replace-role-s0	30.2	17.3	13.8	27.8	16.1	0.66

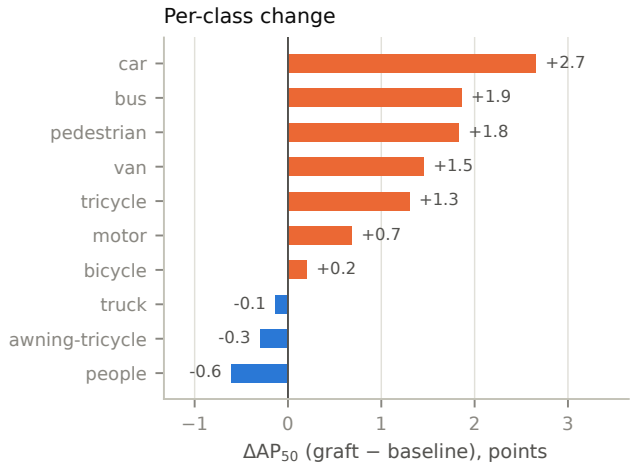


Figure 8: Per-class change in AP_{50} (P2-Graft minus baseline, mean over seeds).